



OASIS Security Report

Cross-Site Scripting (XSS) Security Analysis

Date: 2026-06-12 01:34:12 — **Model:** llama3.2:3b

Vulnerability: Cross-Site Scripting (XSS)

Project: vulnerable

Analysis root: `../vulnerable`

Table of contents

▢ Summary

▢ Statistics

▢ Files

▢ Detailed analysis

▢ Assistant

△ Errors & notes

Summary

Analyzed **3** file(s) for Cross-Site Scripting (XSS).

[Return to table of contents](#)

Statistics

Files analyzed: 3 **Total findings:** 3

Critical



1

High		2
Medium		0
Low		0

[Return to table of contents](#)

Files

File	Similarity
<code>/work/project/dataset/vulnerable/access-control/mycontract.sol</code>	0.650
<code>/work/project/dataset/vulnerable/reentrancy/ 0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol</code>	0.637
<code>/work/project/dataset/vulnerable/reentrancy/etherstore.sol</code>	0.631

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/vulnerable/access-control/mycontract.sol

Similarity score: 0.650

Source lines: 1-31

► Finding 1.1: Cross-Site Scripting (XSS) (High)

Vulnerable code

```
receiver.transfer(amount);
```

Explanation

Unescaped output to HTML can lead to XSS vulnerabilities. The `transfer` function is used directly with user input without proper sanitization.

Impact

Potential session hijacking or credential theft via an intermediary contract

Entry point

/sendTo

Execution path

```
require(tx.origin == owner); receiver.transfer(amount);
```

HTTP methods

POST

Parameters

amount

Exploitation steps

- User sends a malicious tx to the contract with an amount containing JavaScript code

Example payloads

- `javascript:alert('XSS')`

Conditions

Contract has `tx.origin` authorization enabled

Remediation

Use a safer function like ``transfer() { require(tx.origin == owner); }`` or sanitize user input before passing it to the ``transfer()`` function

Secure example

```
require(tx.origin == owner); transfer(amount);  
// Sanitize input  
amount = uint128(unsafe_input);  
require(tx.origin == owner); transfer(amount);
```

File 2: /work/project/dataset/vulnerable/reentrancy/0xbaf51e761510c1a11bf48dd87c0307ac8a8c8a4f.sol

Similarity score: 0.637

Source lines: 1-85

► Finding 2.1: Vulnerability found (Critical)

Vulnerable code

```
function CashOut(uint _am) public payable { if(_am<=balances[msg.sender]) { //  
<yes> <report> REENTRANCY if(msg.sender.call.value(_am>()) { ... } } }
```

Explanation

The use of `msg.sender.call.value()` with user input allows an attacker to inject arbitrary JavaScript code.

Impact

Can lead to session hijacking, credential theft, or delivery of malware to users

Entry point

```
function CashOut(uint _am) public payable { ... }
```

Execution path

```
CashOut(_am)  
- msg.sender.call.value(_am)  
- execution of user-provided code
```

Parameters

`_am`

Exploitation steps

- Get `_am` value from user input

Example payloads

- `malicious JavaScript code`

Conditions

`msg.sender authenticated and balances[msg.sender] > 0`

Remediation

Use `msg.sender.call.value` with strict encoding (e.g. `encode(_am)` instead of `_am`)

Secure example

```
function CashOut(uint _am) public payable { if(_am<=balances[msg.sender])  
{ balances[msg.sender] -= _am; } else { require(_am == 0); } }
```

File 3: /work/project/dataset/vulnerable/reentrancy/etherstore.sol

Similarity score: 0.631

Source lines: 1-39

► Finding 3.1: Cross-Site Scripting (XSS) Vulnerability (High)

Vulnerable code

```
require(msg.sender.call.value(_weiToWithdraw()));
```

Explanation

Using `msg.sender.call.value()` with a user-input variable `_weiToWithdraw` allows an attacker to inject malicious JavaScript code.

Impact

An attacker could steal sensitive data or perform unauthorized actions on behalf of the user.

Entry point

`/withdrawFunds(uint256 _weiToWithdraw)`

Execution path

```
depositFunds -> withdrawFunds -> msg.sender.call.value(_weiToWithdraw())
```

HTTP methods

POST

Parameters

`_weiToWithdraw`

Exploitation steps

- An attacker sends a request to `/withdrawFunds` with a malicious `_weiToWithdraw` value containing JavaScript code.

Example payloads

- `<script>alert('XSS')</script>`

Conditions

The user is authenticated and the withdrawal amount is sufficient.

Remediation

Use `msg.sender.call.value(_weiToWithdraw, 'approved')` instead of `msg.sender.call.value(_weiToWithdraw)`.

Secure example

```
require(msg.sender.call.value(_weiToWithdraw, 'approved'));
```

[Return to table of contents](#)

Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)